

Unit 2: Classes and Objects**2.1 Simple Class, Field****2.2 Access Controls, Object creation****2.3 Construction and initialization****2.4 Inheritance and Polymorphism in Java****2.4.1 Data encapsulation, overriding and overloading methods****2.5 this and super keywords****2.6 Static members, static block, static class****2.7 Interfaces:****2.7.1 Introduction to Interfaces, Interface Declaration.****2.7.2 Inheriting and Hiding concepts.****2.7.3 Inheriting, Overloading and overriding Methods and constructors.****2.7.4 Interfaces Implementation.****2.1 Simple Class and Fields****Definition of a Class**

- A class is a template that specifies the attributes and behavior of things or objects.
- A class is a blueprint or prototype from which objects are created.
- A class is the implementation of an abstract data type (ADT).
- It defines attributes and methods which implement the data structure and operations of the ADT, respectively.

Syntax of a Class

```
class ClassName {  
    Datatype variable1;  
    Datatype variable2;  
    Datatype variableN;  
  
    return_type methodname1 (parameter-list) {  
        // body of method  
    }  
    return_type methodname2(parameter-list) {  
        // body of method  
    }  
    .....  
    return_type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

Key Characteristics

- A class is declared using the class keyword.
 - The data, or variables, defined within a class are called **instance variables**. Each instance of the class (object) contains its own copy of these variables. The data of one object is separate and unique from another object.
 - The methods and variables defined within a class are referred to as **members of the class**.
 - **Methods** contain the actual code for operations.
-

Method Syntax

```
return_type method_name(parameter-list) {  
    // body of method  
}
```

- **return_type**: Specifies the type of data the method returns. If no value is returned, the type must be void.
 - **method_name**: Specifies the name of the method. It must be a unique identifier within the current scope.
 - **parameter-list**: A sequence of type and identifier pairs separated by commas. If no parameters exist, the list is empty.
-

Return Value

- Methods with a return type other than void use the return statement to pass values back to the calling method.

Syntax:

```
return value;
```

- Here, value is the data returned to the calling method.
-

Example: Simple Class

```
class Box {  
    double width = 1;  
    double height = 2;  
    double depth = 3;
```

```
double volume() {  
    return width * height * depth;  
}  
}
```

Method Call Syntax

var_name = object_name.method_name(parameter-list);

Example:

```
Box b1 = new Box(); // Create an object  
double vol = b1.volume(); // Call the method
```

- In the above example, b1 is an object. The method volume() is called and assigned to the variable vol.

2.2 Visibility Controls in Java

Java provides several **access modifiers** to define visibility and access levels for classes, variables, methods, and constructors. The four access levels are:

1. **Package/Friendly (Default)**
 - Visible to all classes within the same package.
 - No explicit modifier is needed.
2. **Private**
 - Visible only within the class where it is declared.
3. **Public**
 - Visible within the class and to all other classes, regardless of package.
4. **Protected**
 - Visible within the same package and accessible by subclasses.

Default Access Modifier (No Keyword)

- This modifier is applied when no access control keyword is specified.
- Members are accessible to all classes within the same package.
- **Note:** Default access cannot be applied to interface methods since they are always public.

Example:

```
String str = "Hi";

void a() {
    System.out.println(str);
}
```

Private Access Modifier (private)

- Methods, variables, and constructors declared private are only accessible within the same class.
- This is the most restrictive access level.
- **Classes and interfaces cannot be declared private.**
- To expose private variables outside the class, **getter and setter methods** must be used.

Example:

```
class A {
    private String s1 = "Hello";

    public String getName() {
        return this.s1;
    }
}
```

- In the above example, s1 is private, meaning it cannot be accessed directly from other classes. The getName() method provides controlled access to it.
-

Public Access Modifier (public)

- The public keyword allows unrestricted access to a class, method, constructor, or variable.
- Members declared public are accessible from any class, even those in different packages.

Example:

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("This is a public method!");
    }
}
```

- The main() method must always be public so that the Java interpreter can call it to start the program.

Protected Access Modifier (protected)

- Protected members can be accessed:
 1. Within the same package.
 2. By subclasses, even if they are in a different package.
- The protected modifier cannot be applied to classes or interfaces.

Example:

```
package p1;

class A {
    float f1;
    protected int i1;
}

package p1;

class B {
    public void getData() {
        A a1 = new A(); // Instance of class A
        a1.f1 = 19;
        a1.i1 = 12; // Accessing protected variable
    }
}
```

- In this example, i1 is a protected variable of class A. It is accessible in class B because they belong to the same package.

2.3 Constructor

Definition

- A **constructor** is a special type of method used to initialize an object.
- It is invoked automatically at the time of object creation.

Rules for Defining a Constructor

1. The constructor name must be the **same** as the class name.
2. A constructor **must not have a return type**, not even void.
3. The return type of a constructor is implicitly the class type itself.

Types of Constructors

1. Default Constructor
 2. Parameterized Constructor
-

1. Default Constructor

- A constructor without any parameters is called a **default constructor**.
- If no constructor is explicitly defined in the class, the compiler automatically provides a default constructor.

Example:

```
class A {  
    A() { // Default constructor  
        System.out.println("Default constructor called..");  
    }  
  
    public static void main(String[] args) {  
        A a = new A();  
    }  
}
```

Output:

Default constructor called..

2. Parameterized Constructor

- A constructor that accepts arguments is called a **parameterized constructor**.
- It allows assigning different values to distinct objects during creation.
- **Note:** If only parameterized constructors are defined, the default constructor will **not** be automatically generated.

Example:

```
class A {  
    int a;  
    String s1;  
  
    A(int b, String s2) { // Parameterized constructor  
        a = b;  
        s1 = s2;  
    }  
}
```

```
}

void display() {
    System.out.println("Value of parameterized constructor is :: " + a + " and " + s1);
}

public static void main(String[] args) {
    A obj = new A(10, "Hello");
    obj.display();
}
}
```

Output:

Value of parameterized constructor is :: 10 and Hello

3. Copy Constructor

- A **copy constructor** is a constructor that takes one parameter of the same class type.
- It is used to create a new object as a copy of an existing object.
- The copy is independent of the original object, occupying a different memory address.

Example:

```
class Box {
    double width, height, depth;

    // Copy constructor
    Box(Box ob) {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
}
```

4. Overloaded Constructors

- Constructor overloading allows defining multiple constructors within a class, each with a different signature.
- Unlike methods, constructors do not have a return type.

Passing Object as a Parameter

- To construct a new object initially identical to an existing object, define a constructor that takes an object of the same class as a parameter.

Example:

```
class Box {
    double width, height, depth;

    // Copy constructor
    Box(Box ob) {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
    // Parameterized constructor
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
    // Default constructor
    Box() {
        width = height = depth = -1; // Indicating uninitialized box
    }
    // Constructor for cube
    Box(double len) {
        width = height = depth = len;
    }

    // Calculate and return volume
    double volume() {
        return width * height * depth;
    }
}

class DemoAllCons {
    public static void main(String[] args) {
        Box mybox1 = new Box(10, 20, 15); // Parameterized
        Box mybox2 = new Box();           // Default
        Box mycube = new Box(7);          // Cube
        Box myclone = new Box(mybox1);    // Copy constructor

        System.out.println("Volume of mybox1 is " + mybox1.volume());
    }
}
```



```

        System.out.println("Volume of mybox2 is " + mybox2.volume());
        System.out.println("Volume of cube is " + mycube.volume());
        System.out.println("Volume of clone is " + myclone.volume());
    }
}

```

Output:

Volume of mybox1 is 3000.0
 Volume of mybox2 is -1.0
 Volume of cube is 343.0
 Volume of clone is 3000.0

2.4 Inheritance**Definition**

- **Inheritance** is the process by which a class can acquire the properties and methods of its parent class.
- A **child class** (derived class) is created from a **parent class** (base class).
- Inheritance enables:
 - **Reusability:** Methods and fields from the parent class can be reused.
 - **Extensibility:** New methods and fields can be added to the child class.
- All properties of a superclass, except private members, can be inherited in a subclass using the extends keyword.
- The inherited fields can be used directly, just like any other fields. You can declare a field in the subclass with the same name as the one in the superclass, thus hiding it (not recommended).
- You can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- You can write a new instance method in the subclass that has the same signature as the one in the superclass, thus overriding it.

Types of Inheritance**Single Inheritance:**

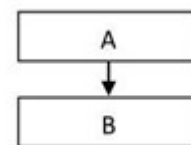
A class is derived from a single parent class.

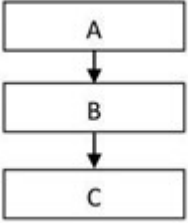
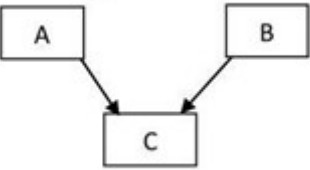
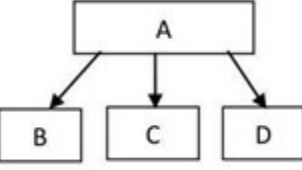
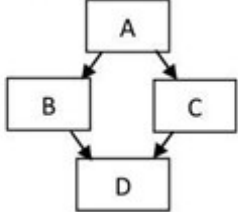
Example:

```

class A {
    public void displayA() {
        System.out.println("Class A method");
    }
}
class B extends A {
    public void displayB() {
        System.out.println("Class B method");
    }
}

```



Multilevel Inheritance: A class is derived from a class that is itself derived from another class. Example: class A { ... } class B extends A { ... } class C extends B { ... }	 <pre> graph TD A[A] --> B[B] B --> C[C] </pre>
Multiple Inheritance: Not supported directly in Java through classes to avoid ambiguity.	 <pre> graph TD A[A] --> C[C] B[B] --> C </pre>
Hierarchical Inheritance: Multiple classes are derived from a single parent class. Example: class A { ... } class B extends A { ... } class C extends A { ... }	 <pre> graph TD A[A] --> B[B] A --> C[C] A --> D[D] </pre>
Hybrid Inheritance: Combination of single and multiple inheritance. Not directly supported in Java.	 <pre> graph TD A[A] --> B[B] A --> C[C] B --> D[D] C --> D </pre>

Example

```

class A {
    public void displayA() {
        System.out.println("Class A method");
    }
}

class B extends A { // Single Inheritance
    public void displayB() {
        System.out.println("Class B method");
    }
}

class C extends B { // Multilevel Inheritance
    public void displayC() {
        System.out.println("Class C method");
    }
}

```

```
class D extends A { // Hierarchical Inheritance
    public void displayD() {
        System.out.println("Class D method");
    }
}

class Trial {
    public static void main(String[] args) {
        B b = new B();
        C c = new C();
        D d = new D();

        b.displayB();
        c.displayC();
        d.displayD();
    }
}
```

2.4.1 Encapsulation

Definition

- Encapsulation is the mechanism of binding together data and the methods that manipulate it, hiding implementation details from the outside world.
- It combines **data hiding** and **abstraction**.
- **Achieved by:**
 - Declaring variables as private.
 - Providing public getter and setter methods for controlled access.

Example:

```
class Person {
    private String name;
    private int age;

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
}

class Main {
    public static void main(String[] args) {
        Person person = new Person();
    }
}
```

```
    person.setName("John");
    person.setAge(30);

    System.out.println("Name: " + person.getName());
    System.out.println("Age: " + person.getAge());
}
}
```

Advantages:

- Data Hiding
- Increased Flexibility
- Reusability
- Easier Testing

2.4.2 Method Overriding**Definition**

- Occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.
- Enables **runtime polymorphism**.

Example:

```
class A {
    public void display() {
        System.out.println("Class A");
    }
}

class B extends A {
    public void display() {
        System.out.println("Class B");
    }
}

class Trial {
    public static void main(String[] args) {
        A a = new A();
        B b = new B();

        a.display(); // Output: Class A
        b.display(); // Output: Class B
    }
}
```

Dynamic Method Dispatch

Definition

- Also known as **runtime polymorphism**, it is the process of resolving method calls at runtime based on the object being referenced.

Example:

```
class A {
    void callme() {
        System.out.println("Inside A's callme method");
    }
}
class B extends A {
    void callme() {
        System.out.println("Inside B's callme method");
    }
}

class DispatchDemo {
    public static void main(String[] args) {
        A obj; // Reference of type A

        obj = new A();
        obj.callme(); // Output: Inside A's callme method

        obj = new B();
        obj.callme(); // Output: Inside B's callme method
    }
}
```

2.4.1 Method Overloading

Definition

- Method Overloading** occurs when a class has multiple methods with the **same name** but **different parameters**.
- It is also known as **compile-time (static) polymorphism**.

Characteristics

- The same method name can be used with:
 - Different numbers of parameters**
 - Different data types of parameters**
 - Different order of parameters**

- **Overloading methods with different return types is not allowed.**
 - During compilation, the **compiler determines** which method to call based on the type and number of arguments passed.
-

Example:

```
class OverloadingDemo {  
    // Method with two integer parameters  
    void sum(int a, int b) {  
        System.out.println("Sum of (a+b) is:: " + (a + b));  
    }  
  
    // Method with three integer parameters  
    void sum(int a, int b, int c) {  
        System.out.println("Sum of (a+b+c) is:: " + (a + b + c));  
    }  
  
    // Method with two double parameters  
    void sum(double a, double b) {  
        System.out.println("Sum of double (a+b) is:: " + (a + b));  
    }  
  
    public static void main(String[] args) {  
        OverloadingDemo obj = new OverloadingDemo();  
  
        obj.sum(10, 10);    // Calls the method with two integer parameters  
        obj.sum(10, 10, 10); // Calls the method with three integer parameters  
        obj.sum(10.5, 10.5); // Calls the method with two double parameters  
    }  
}
```

Output

```
Sum of (a+b) is:: 20  
Sum of (a+b+c) is:: 30  
Sum of double (a+b) is:: 21.0
```

this Keyword

- Refers to the current object of the class.
- Use of this keyword
 1. **Distinguishing Instance Variables:** Resolves ambiguity when instance variables are shadowed by method parameters or local variables.

```
class Box {  
    int width;  
    Box(int width) {  
        this.width = width; // `this` refers to the instance variable  
    }  
}
```

2. **Calling Other Constructors:** Used to call another constructor within the same class.

```
class Box {  
    int width, height;  
    Box(int width) {  
        this(width, 10); // Calls the two-parameter constructor  
    }  
    Box(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
}
```

3. **Returning the Current Object:** Allows methods to return the current instance for method chaining.

```
class Box {  
    Box setWidth(int width) {  
        this.width = width;  
        return this;  
    }  
}
```

super Keyword

Refers to the immediate parent class object.

Use of super keyword

1. **Accessing Parent Class Members:** Access parent class fields or methods overridden in the child class.

```
class Parent {  
    String name = "Parent";  
}  
class Child extends Parent {
```

```
String name = "Child";
void display() {
    System.out.println(super.name); // Refers to the parent class field
}
}
```

2. **Calling Parent Class Methods:** Used to invoke overridden methods in the parent class.

```
class Parent {
    void display() {
        System.out.println("Parent Method");
    }
}
class Child extends Parent {
    void display() {
        super.display(); // Calls the parent class method
        System.out.println("Child Method");
    }
}
```

3. **Calling Parent Class Constructors:** Explicitly calls a parameterized or default constructor of the parent class.

```
class Parent {
    Parent(String message) {
        System.out.println("Parent Constructor: " + message);
    }
}
class Child extends Parent {
    Child(String message) {
        super(message); // Calls the parent class constructor
    }
}
```

Key Differences

Feature	this	super
Refers To	Current object	Parent class object
Access Scope	Members of the same class	Members of the immediate parent class
Constructor Usage	Calls another constructor in the same class	Calls a constructor of the parent class
Overridden Method Access	Not applicable	Access overridden methods

These keywords are essential for managing class hierarchies and handling object-specific references in Java programming.

Difference between Method Overloading and Overriding

Aspect	Overriding	Overloading
Argument List	The argument list must exactly match that of the overridden method.	Overloaded methods MUST change the argument list.
Return Type	The return type must be the same as the overridden method in the super class.	Overloaded methods CAN change the return type.
Private and Final Methods	Private and final methods cannot be overridden.	Private and final methods can be overloaded.
Occurs In	Method overriding occurs in two classes that have inheritance.	Method overloading is performed within a class.
Binding Type	Dynamic binding is used for overridden methods.	Static binding is used for overloaded methods.
Polymorphism Type	Method overriding is the example of run-time polymorphism.	Method overloading is the example of compile-time polymorphism.

Static Keyword in Java

The static keyword in Java is primarily used for memory management. It allows variables and methods to be shared across all instances of a class, meaning they belong to the class rather than any particular object of the class. This is useful for creating constants or methods that should be the same for every instance of the class.

Key Applications:

- **Variables**
- **Methods**
- **Blocks**
- **Nested Classes**

The static keyword is a **non-access modifier** that is applied as follows:

- To create a static member (block, variable, method, nested class), prefix the declaration with static.

Characteristics of the static Keyword:

1. **Shared Memory Allocation:** Static variables and methods are allocated memory space only once, shared by all instances of the class. This is useful for global states or shared functionalities.
 2. **Accessible Without Object Instantiation:** Static members can be accessed without creating an instance of the class. This is ideal for utility functions and constants.
 3. **Associated with the Class:** Static members are linked to the class itself, not to specific objects. Changes to static members are reflected in all instances of the class.
 4. **Cannot Access Non-Static Members:** Static methods cannot directly access non-static members of the class as they are not associated with any particular instance.
 5. **Overloadable but Not Overridable:** Static methods can be overloaded (same method name with different parameters), but they cannot be overridden since they belong to the class, not an object.
 6. **Access Before Instantiation:** A static member can be accessed before any objects of its class are created.
-

Static Blocks:

Static blocks are used for initialization of static variables and are executed once when the class is loaded.

Example:

```
class Test {  
    static int a = 10;  
    static int b;  
  
    static {  
        System.out.println("Static block initialized.");  
        b = a * 4;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("from main");  
        System.out.println("Value of a : " + a);  
        System.out.println("Value of b : " + b);  
    }  
}
```

Output:

```
Static block initialized.  
from main  
Value of a : 10  
Value of b : 40
```

Static Variables:

Static variables are shared across all instances of the class, essentially acting as global variables for that class.

Example:

```
class Test {  
    static int a = m1();  
  
    static {  
        System.out.println("Inside static block");  
    }  
  
    static int m1() {  
        System.out.println("from m1");  
        return 20;  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Value of a : " + a);  
        System.out.println("from main");  
    }  
}
```

Output:

```
from m1  
Inside static block  
Value of a : 20  
from main
```

Static Methods:

Static methods are methods that belong to the class rather than any instance. They can only access static variables and other static methods.

Restrictions:

1. Static methods can only directly call other static methods.
2. Static methods cannot access instance variables or methods directly.
3. Static methods cannot use this or super.

Example of Restrictions:

```
class Test {  
    static int a = 10;
```

```
int b = 20;

static void m1() {
    a = 20;
    System.out.println("from m1");

    b = 10; // Error: Cannot access non-static field 'b'
    m2();   // Error: Cannot call non-static method 'm2'
    System.out.println(super.a); // Error: Cannot use 'super' in static context
}

void m2() {
    System.out.println("from m2");
}

public static void main(String[] args) {
    // Main method
}
}
```

Use Cases for Static Variables and Methods:

1. **Static Variables:** For properties that are common to all instances, like a shared constant or counter.
2. **Static Methods:** For utility functions that don't require object-specific data.

Example:

```
class Student {
    String name;
    int rollNo;

    static String clgName;
    static int counter = 0;

    public Student(String name) {
        this.name = name;
        this.rollNo = setRollNo();
    }

    static int setRollNo() {
        counter++;
        return counter;
    }
}
```

```
static void setCllg(String name) { cllgName = name; }

void getStudentInfo() {
    System.out.println("name : " + this.name);
    System.out.println("rollNo : " + this.rollNo);
    System.out.println("cllgName : " + cllgName);
}
}

public class StaticDemo {
    public static void main(String[] args) {
        Student.setCllg("XYZ");

        Student s1 = new Student("Alice");
        Student s2 = new Student("Bob");

        s1.getStudentInfo();
        s2.getStudentInfo();
    }
}
```

Output:

```
name : Alice
rollNo : 1
cllgName : XYZ
name : Bob
rollNo : 2
cllgName : XYZ
```

Static Classes:

A class can only be declared static if it is a nested class. A static nested class does not require a reference to the outer class and cannot access non-static members of the outer class.

Example of Static Nested Class:

```
public class EX_PROGRAM {
    private static String str = "Java programming language";

    static class MyNestedClass {
        public void disp() {
            System.out.println(str);
        }
    }
}
```

```
public static void main(String args[]) {  
    EX_PROGRAM.MyNestedClass obj = new EX_PROGRAM.MyNestedClass();  
    obj.disp();  
}  
}
```

Output:

Java programming language

Advantages of Static Members:

1. **Memory Efficiency:** Static members are allocated memory only once, saving memory.
2. **Improved Performance:** Static members can be accessed more efficiently.
3. **Global Accessibility:** Static members can be accessed from anywhere in the program.
4. **Encapsulation of Utility Methods:** Static methods encapsulate functions that don't require object state.
5. **Constants:** Static final variables can define constants used across the program.
6. **Class-Level Functionality:** Static methods can define functionality that doesn't depend on object state.

Interfaces in Java

An **interface** in Java is an abstract type used to define the behavior of a class. It acts as a **blueprint** for behavior, specifying what methods a class must implement but without defining how those methods are implemented. Java interfaces are primarily used to achieve **abstraction** and **multiple inheritance**.

- An **interface** defines a contract that classes must adhere to by implementing the methods declared in the interface.
- **Interfaces** enable **abstraction** and help achieve **multiple inheritance** in Java.
- They are used to define common behaviors and reduce dependencies, allowing for more flexible and modular code.

What are Interfaces in Java?

An interface in Java is a mechanism to achieve abstraction. It was traditionally used to declare **abstract methods** (methods without a body) and **public, static, final variables** by default. The key features of an interface are:

1. **Abstraction:** Interfaces define the behavior of a class without providing the implementation details.
2. **Multiple Inheritance:** Unlike classes, a class can implement multiple interfaces, which helps achieve multiple inheritances in Java.
3. **IS-A Relationship:** Interfaces are often used to define a relationship where a class "is a" type of interface.

In simpler terms, interfaces are used to define what a class can do, without dictating how it does it.

Syntax of Java Interface

To declare an interface, use the **interface** keyword. Here's the basic structure:

```
interface InterfaceName {  
    // Declare constant fields (public, static, final by default)  
    // Declare abstract methods (public by default)  
    int constant = 100;  
  
    void method1(); // abstract method  
    void method2(); // abstract method  
}
```

- **Constant fields** in interfaces are **public**, **static**, and **final** by default.
- **Methods** in an interface are **abstract** and **public** by default. They do not have a body.

How to Implement an Interface

A class that implements an interface must provide the implementation for all the methods declared in that interface. To implement an interface, the class uses the **implements** keyword.

```
class MyClass implements InterfaceName {  
    public void method1() {  
        System.out.println("method1 implemented");  
    }  
    public void method2() {  
        System.out.println("method2 implemented");  
    }  
}
```

In this example, **MyClass** provides implementations for the methods defined in **InterfaceName**.

Uses of Interfaces in Java

Interfaces serve several important purposes in Java:

1. **Achieving Abstraction:** Interfaces allow for total abstraction by defining methods without providing implementation.
 2. **Multiple Inheritance:** Java does not support multiple inheritance with classes, but a class can implement multiple interfaces, thus achieving multiple inheritance.
 3. **Loose Coupling:** Interfaces help in reducing the dependencies between classes, enabling loose coupling.
 4. **Defining Common Behavior:** Interfaces can define common behavior for classes to implement.
-

Why Use Interfaces Instead of Abstract Classes?

While both interfaces and abstract classes are used for abstraction, there are key differences:

- **Abstract classes** can have non-final variables, while **interface variables** are always **final**, **public**, and **static**.
- **Abstract classes** can have method implementations, but **interfaces** only declare abstract methods (though in Java 8 and later, interfaces can also have default and static methods with implementations).

Example:

```
// Simple Interface Example
interface Player {
    int id = 10; // By default, public, static, final
    int move(); // Abstract method
}
```

Relationship Between Class and Interface

1. **Classes and Interfaces:** A class can extend another class and implement one or more interfaces. However, an interface can extend other interfaces, but it cannot implement a class.
 2. **Multiple Interfaces:** A class can implement multiple interfaces, allowing a form of multiple inheritance.
 3. **IS-A Relationship:** When a class implements an interface, it forms an **IS-A** relationship, meaning the class can be treated as the interface type.
-

Difference Between Class and Interface

Although both **Classes** and **Interfaces** are used in Java to define types, they have some key differences. Here's a comparison of the major differences:

- **Classes** are used to define objects with both data and methods, and they allow for full implementation.
- **Interfaces** are used to define a contract that classes must follow, specifying behavior without implementation details.

Class	Interface
Instantiation: A class can have instance variables and you can create objects from it.	Instantiation: In an interface, variables must be initialized (as they are final by default), but you cannot create objects from an interface.
Methods: A class can contain concrete methods (methods with implementations).	Methods: An interface can only declare abstract methods (methods without implementation). In Java 8 and later, interfaces can have default or static methods with implementation.
Access Specifiers: Classes can use private, protected, and public access specifiers for methods and variables.	Access Specifiers: In interfaces, methods and variables are public by default, and no other access modifiers (like private or protected) can be used.

Implementing Interfaces in Java

To implement an interface in Java, we use the implements keyword. Below is a simple example demonstrating how interfaces work in Java.

- **Implementing Interfaces:** Use the implements keyword to implement an interface in Java.
- **Advantages:** Interfaces help in achieving abstraction, multiple inheritance, and loose coupling.
- **Multiple Inheritance:** Interfaces allow multiple inheritance, unlike classes.
- **Extending Interfaces:** Interfaces can inherit other interfaces using the extends keyword, and a class must implement all the methods from the inherited interfaces.

1. Simple Interface Example

```
import java.io.*;

// A simple interface
```

```
interface In1 {  
    // public, static, and final variable  
    final int a = 10;  
  
    // public and abstract method  
    void display();  
}  
  
// A class that implements the interface  
class TestClass implements In1 {  
    // Implementing the capabilities of the interface  
    public void display() {  
        System.out.println("Hello");  
    }  
  
    // Driver Code  
    public static void main(String[] args) {  
        TestClass t = new TestClass();  
        t.display();  
        System.out.println(t.a);  
    }  
}
```

Output:

Hello
10

Advantages of Interfaces in Java

Using interfaces in Java offers several advantages:

- **Security of Implementation:** Interfaces allow us to define the behavior of a class without worrying about the implementation details.
 - **Achieving Multiple Inheritance:** While Java does not support multiple inheritance through classes, interfaces can be used to achieve multiple inheritance by allowing a class to implement multiple interfaces.
-

2. Multiple Inheritance Using Interfaces

Java doesn't support multiple inheritance with classes, but interfaces allow us to implement multiple inheritance. Here's an example:

```
import java.lang.*;
import java.io.*;
interface Exam
{
    void percent_cal();
}
class Student
{
    String name;
    int roll_no, mark1, mark2;
    Student(String n, int r, int m1, int m2)
    {
        name=n;
        roll_no=r;
        mark1=m1;
        mark2=m2;
    }
    void display()
    {
        System.out.println ("Name of Student: "+name);
        System.out.println ("Roll No. of Student: "+roll_no);
        System.out.println ("Marks of Subject 1: "+mark1);
        System.out.println ("Marks of Subject 2: "+mark2);
    }
}
class Result extends Student implements Exam
{
    Result(String n, int r, int m1, int m2)
    {
        super(n,r,m1,m2);
    }
    public void percent_cal()
    {
        int total=(mark1+mark2);
        float percent=total*100/200;
```

```
        System.out.println ("Percentage: "+percent+"%");
    }
    void display()
    {
        super.display();
    }
}
class Multiple_Inheritance
{
    public static void main(String args[])
    {
        Result R = new Result("Ra.one",12,93,84);
        R.display();
        R.percent_cal();
    }
}
```

Output:

D:\javap>javac Multiple_Inheritance.java

D:\javap>java Multiple_Inheritance

Name of Student: Ra.one

Roll No. of Student: 12

Marks of Subject 1: 93

Marks of Subject 2: 84

Percentage: 88.0%

3. Extending Interfaces

In Java, one interface can inherit from another using the **extends** keyword. When a class implements an interface that inherits another interface, it must provide an implementation for all methods required by the inheritance chain.

Program 1:

```
interface A {
    void method1();
    void method2();
}
// B inherits methods from A
interface B extends A {
    void method3();
}
```

```
// Class implements methods from both A and B
class gfg implements B {
    public void method1() {
        System.out.println("Method 1");
    }
    public void method2() {
        System.out.println("Method 2");
    }
    public void method3() {
        System.out.println("Method 3");
    }
}
```

Program 2:

```
interface Student {
    void data();
}
class avi implements Student {
    public void data() {
        String name = "Ram";
        int rollno = 68;
        System.out.println(name);
        System.out.println(rollno);
    }
}
public class InterfaceExample {
    public static void main(String[] args) {
        avi h = new avi();
        h.data();
    }
}
```

Output:

```
Ram
68
```

Interface and Abstract Class

- **Interfaces** are ideal for defining a contract (a set of methods that must be implemented by a class) and allow for multiple inheritance.
- **Abstract classes** are ideal for providing a common base with shared code and can have both abstract and concrete methods, making them more flexible than interfaces.

When to Use Interface vs Abstract Class?

- **Use an Interface** when:
 - You need to define a contract that classes must follow (e.g., multiple unrelated classes implementing the same set of methods).
 - You want to achieve multiple inheritance of behaviors.
 - You want to achieve total abstraction (no implementation in the interface).
- **Use an Abstract Class** when:
 - You have a base class that shares common code (state and behavior) among several related subclasses.
 - You need to provide some default behavior (concrete methods) along with abstract methods.
 - You want to use fields and constructors in the base class.

Differences Between Interface and Abstract Class

Feature	Interface	Abstract Class
Methods	Can only have abstract methods (until Java 8).	Can have both abstract and concrete methods.
Implementation	Classes must implement all abstract methods.	Subclasses can implement some abstract methods.
Multiple Inheritance	Supports multiple inheritance (via interfaces).	Only supports single inheritance.
Constructors	Cannot have constructors.	Can have constructors.
Fields	Can only have public static final variables.	Can have instance variables with any access modifier.
Access Modifiers	All methods are implicitly public.	Can have public, private, or protected methods.
Use Case	Used to represent behaviors that can be implemented by any class.	Used when classes share common state or behavior.